ZipMould

Um *solver* para *puzzles* Zip inspirado em *slime mould*

Gabriel Mitelman Tkacz · Maio de 2026

Da metaheurística contínua à busca combinatória discreta — adaptando **Li et al. (2020)** para um domínio de caminho Hamiltoniano.

Roteiro: três atos em vinte minutos

Ato 1 · Li (≈7 min)

O que é o **Slime Mould Algorithm**?
A biologia, as equações e a evidência empírica.

Ato 2 · ZipMould (≈7 min)

Como adaptar uma SMA contínua a um problema **combinatório discreto**. Onde a analogia funciona, onde ela quebra e o que precisou mudar.

Ato 3 · *Demo* (≈5 min)

Reprodução de *trace* ao vivo no **visualizador Vue 3**.

A intuição em uma linha

A atualização de Li

$$X(t+1) = v_c \cdot X(t) + v_b \cdot (W \cdot X_A - X_B)$$

vira a atualização de arestas do ZipMould

$$\tau \leftarrow v_c \cdot \tau + v_b \cdot \Delta_{\textit{rank-weighted}}$$

O que é *Physarum polycephalum*?

- ***Slime mould* acelular.** Um único organismo multinucleado, sem sistema nervoso.
- Procura alimento estendendo uma **rede venosa** pelo substrato.
- ***Feedback* positivo**: mais alimento → fluxo citoplasmático mais rápido → veias mais grossas.
- ***Feedback* negativo**: ramos sem alimento retraem.
- Já foi usado para resolver labirintos, aproximar a **rede ferroviária de Tóquio** e agir como um otimizador distribuído sem controle central.

> A SMA transforma três comportamentos observados em regras de atualização: ***approaching food***, ***wrapping food*** (*feedback* na espessura das veias) e ***grabbling food*** (amplitude do bio-oscilador).

Li, Chen, Wang, Heidari & Mirjalili (2020), Future Generation Computer Systems 111, 300–323. Tero et al. (2010), Science 327, 439–442.

SMA Eq. (2.1) — *aproximar-se do alimento*

$$\vec{X}(t+1) = \begin{cases} \vec{X}_b(t) + \vec{v}_b \cdot \left(\vec{W} \cdot \vec{X}_A(t) - \vec{X}_B(t)\right), & r < p \\ \vec{v}_c \cdot \vec{X}(t), & r \geq p \end{cases}$$

- $\vec{X}_b$ — melhor indivíduo encontrado até agora.
- $\vec{X}_A, \vec{X}_B$ — dois indivíduos aleatórios que definem uma direção de exploração.
- $\vec{W}$ — **peso** derivado do *ranking* de *fitness* (Eq. 2.5, próximo *slide*).
- $\vec{v}_b \in [-a, a]$, em que $a = \operatorname{arctanh}(1 - t/T)$.
- $\vec{v}_c$ decresce linearmente de 1 para 0.
- $p = \tanh|S(i) - DF|$ — limiar adaptativo para alternar o comportamento.

Li et al. (2020), §2.3.1.

SMA Eq. (2.5) — o peso $\vec{W}$

$$W_i = \begin{cases} 1 + r \cdot \log\left(\dfrac{bF - S(i)}{bF - wF} + 1\right), & i \in \text{metade superior (boa avaliação)} \\ 1 - r \cdot \log\left(\dfrac{bF - S(i)}{bF - wF} + 1\right), & i \in \text{metade inferior (avaliação ruim)} \end{cases}$$

- A **metade superior** da população puxa para áreas favoráveis — ***feedback* positivo**.
- A **metade inferior** é empurrada para longe — ***feedback* negativo** simulando a retração de veias sem alimento.
- $\log$ suaviza a taxa de mudança; $r \sim \mathcal{U}(0, 1)$ mantém a resposta estocástica.
- Captura a "preferência" do *slime mould* pelo ***ranking* de *fitness***, não pela *fitness* absoluta.

Li et al. (2020), §2.3.2 — modelo matemático de "Wrap food".

SMA Eq. (2.7) — a atualização completa

$$\vec{X}^* = \begin{cases} \text{rand} \cdot (UB - LB) + LB, & \text{rand} < z \\ \vec{X}_b(t) + \vec{v}_b \cdot (W \cdot \vec{X}_A - \vec{X}_B), & r < p \\ \vec{v}_c \cdot \vec{X}(t), & r \geq p \end{cases}$$

- Três casos, **por indivíduo, por iteração**:
  i. ***z-branch*** (prob. $z = 0.03$ em Li): reinício aleatório no espaço de busca.
  ii. Caso ***approach*** (prob. $\approx p$): explora a melhor solução com perturbação ponderada por W.
  iii. Caso ***oscillate***: contrai a solução em direção à origem com v_c.

A *z-branch* permite que a SMA *escape de ótimos locais* sem *niching* explícito e sem reiniciar a população inteira.

Li et al. (2020), §2.3.2 Eq. (2.7); z escolhido como 0.03 a partir do sweep de sensibilidade §3.4.

A evolução de v_b / v_c

v_c — contração linear

$$v_c \in [-1, 1], \quad v_c \to 0 \text{ quando } t \to T$$

Amortece suavemente o caso *oscillate*. No fim da execução, $v_c \cdot X \approx 0$: o agente praticamente para de se mover sozinho.

v_b — amplitude saturante

$$v_b \in [-a, a], \quad a = \operatorname{arctanh}(1 - t/T)$$

No início: $a \to \infty$ → saltos grandes. No fim: $a \to 0$ → exploração local mais fina.

Efeito combinado

Iterações iniciais: ***exploration*** domina porque v_b é grande.

Iterações finais: ***exploitation*** domina quando v_b e v_c encolhem.

O *slime mould* "decide se aproxima da fonte atual ou procura outra" — aqui, isso aparece como amplitude de oscilação.

Li et al. (2020), §2.3.3 "Grabble food", Figura 5.

SMA — o algoritmo

```
INICIALIZAR população X_1 ... X_n aleatoriamente em [LB, UB]
PARA t = 1 ... T:
    avaliar aptidão S(i) para todo i
    ordenar população, identificar bF, wF, X_b
    calcular W via Eq. (2.5)       # pesos positivos/negativos por ordenação
    PARA cada indivíduo i:
        amostrar r ~ U(0,1), rand ~ U(0,1)
        atualizar v_b, v_c, p       # evolução
        SE rand < z:
            X_i ← reinício aleatório em [LB, UB]
        SENÃO SE r < p:
            X_i ← X_b + v_b * (W * X_A - X_B)   # aproximação
        SENÃO:
            X_i ← v_c * X_i                     # oscilação
RETORNAR bF, X_b
```

- Um *loop* externo, três casos internos, sem derivadas e sem gradientes.
- Cinco hiperparâmetros no total: população n, iterações T, probabilidade de reinício z e as constantes embutidas em v_b, v_c.

Por que isso chamou atenção

Caso empírico

- **23 *benchmarks* clássicos** (unimodais + multimodais) + **10 funções CEC2014**: SMA vence ou empata em primeiro na maioria.
- Supera WOA, GWO, MFO, BA, SCA, PSO, SSA, MVO, ALO na maior parte dos casos multimodais.
- **4 problemas de projeto de engenharia** (viga soldada, vaso de pressão, *cantilever*, *I-beam*): melhor solução viável nos quatro.
- As curvas de convergência mostram **queda inicial rápida + refinamento final preciso**.

Li et al. (2020), Tabelas 5–22; Figuras 9–14 (curvas de convergência).

Por que funciona

- W implementa um **termo de diversidade** explícito — a repulsão da metade inferior evita convergência prematura.
- A evolução de v_b cria uma transição automática ***exploration→exploitation***, sem agenda externa de operadores.
- A fuga via *z-branch* é **simples, mas efetiva** para sair de bacias locais.

O *puzzle* Zip

Definição formal

Dada uma grade $G_{N \times N}$ com:

- K *waypoints* ordenados $w_1, w_2, \ldots, w_K$
- Um conjunto de **paredes** (arestas proibidas entre células adjacentes)
- Um conjunto de células **bloqueadas**

Queremos encontrar um **caminho Hamiltoniano** $\pi_1, \ldots, \pi_L$ (em que $L = N^2 - |\text{bloqueadas}|$) tal que:

1. células consecutivas sejam 4-adjacentes e não separadas por parede,
2. *waypoints* apareçam em **ordem crescente**,
3. $\pi_1 = w_1$ e $\pi_L = w_K$.

O *puzzle* Zip diário do LinkedIn popularizou o formato.

O problema de decisão é **NP-completo** (reduz a *Hamiltonian-path-with-pinned-vertices*), mas é tratável na prática para $N \leq 10$ — exatamente a faixa em que metaheurísticas começam a valer a pena.

Por que não dá para aplicar a SMA contínua diretamente

A SMA de Li vive em $\mathbb{R}^d$

- $\vec{X}_A - \vec{X}_B$ é um vetor de direção euclidiano.
- $v_b \cdot W$ escala uma amplitude em espaço contínuo.
- O passo é apenas soma vetorial.

Zip é um problema em grafo

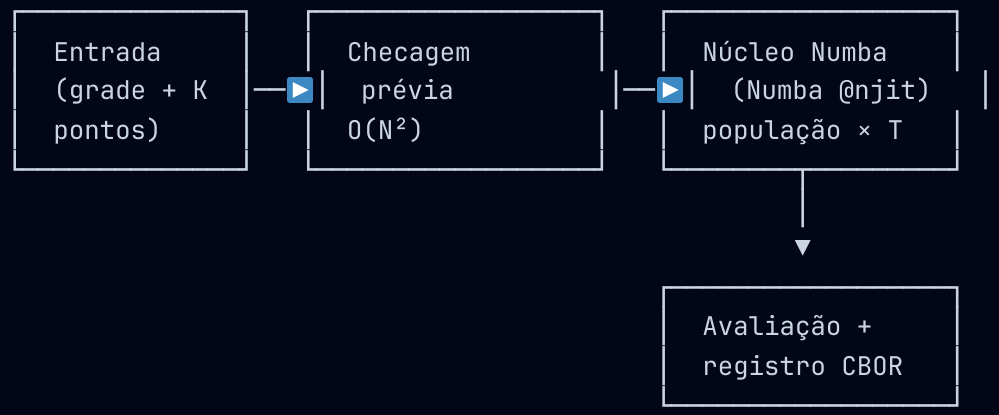
- "Posição" é um caminho Hamiltoniano parcial, não uma coordenada.
- $\vec{X}_A - \vec{X}_B$ é **indefinido** entre dois caminhos.
- O estado natural é **uso de arestas**, não coordenadas de pontos.

A ponte

A saída é usar ***stigmergy***, como em **Ant Colony Optimisation** (Dorigo, 1992): o feromônio τ nas arestas passa a ser o estado do agente.

Em seguida, levamos a **dinâmica de atualização** da SMA — evolução de v_b/v_c, pesos assinados por *ranking* e *z-restart* — para o feromônio, não para uma coordenada.

ZipMould — *pipeline*



Checagens de viabilidade (baratas e decisivas)

- *Waypoint* alcançável e não bloqueado
- Subgrafo livre **conectado** (BFS a partir de w_1 cobre todas as células livres)
- **Limite de paridade**: $|F_0 - F_1| \leq 1$ na coloração de tabuleiro
- **Paridade dos extremos** consistente com w_1, w_K

Se qualquer uma falha, o *puzzle* é inviável — e o *kernel* nem roda.

Etapa 1 — construção estilo ACO

Cada *walker* escolhe um vizinho 4-adjacente via ***softmax*(feromônio + heurística)**:

$$P(c \to c') \propto \exp\Big(\alpha \cdot \tau_{cc'} + \beta \cdot \log \eta_{c'}\Big)$$

Heurística combinada:

$$\eta_{c'} = \mathrm{softplus}(h_m)^{\gamma_m} \cdot \mathrm{softplus}(h_w)^{\gamma_w} \cdot \mathrm{softplus}(h_a)^{\gamma_a} \cdot \mathrm{softplus}(h_p)^{\gamma_p}$$

Heurística	Papel	Fonte
h_m — Manhattan	Puxar para o próximo *waypoint*	$-d_M(c', w_{\mathrm{seg}+1})$
h_w — Warnsdorff	Preferir baixo grau; consumir becos sem saída cedo	heurística *knight's tour* (1823)
h_a — Articulação	Rejeitar movimentos que **desconectam** o subgrafo livre não visitado	checagem *flood-fill*
h_p — Paridade	Manter $\lvert F_0 - F_1 \rvert \leq 1$ após o movimento	invariante de tabuleiro

A combinação por softplus *aceita sinais mistos;* defaults *α = 1, β = 2 seguem ACO (Dorigo & Stützle, 2004).*

Etapa 2 — atualização do feromônio no estilo SMA

```
# src/zipmould/solver/_kernel.py — _pheromone_update
progress = float(t) / float(T)
v_b = math.tanh(1.0 - progress)        # inspirado em Li, LIMITADO (cf. arctanh)
v_c = 1.0 - progress                   # Li 2.4 literal

# Pesos assinados por ordenação ∈ [-1, +1]: melhor agente = +1, pior = -1, mediana ≈ 0
denom = float(n - 1)
weights[i] = (float(n) - 2.0 * float(r) + 1.0) / denom

# Atualização por aresta — o análogo ZipMould da Eq. (2.7) de Li
new_val = v_c * tau[s, e] + v_b * deposit[s, e]

# Escape por reinício z de Li — literal, nas arestas
if z > 0.0 and np.random.random() < z:
    new_val = np.random.normal(0.0, tau_max / 4.0)
```

O peso **assinado** por *ranking* é o análogo discreto do W de Li: *walkers* da metade superior *depositam* feromônio; os da metade inferior o *evaporam* nas mesmas arestas. Sem sinal, o método vira ACO *vanilla*.

O que mudou em relação a Li, e por quê

Li (2020) — contínuo	ZipMould — discreto	Por que a mudança
Estado $\vec{X} \in \mathbb{R}^d$	Feromônio $\tau \in \mathbb{R}^m$ (m = #arestas)	Não há espaço de coordenadas; arestas carregam memória
$W_i = 1 \pm r \log(\cdot)$	$W_i = (n - 2r + 1)/(n - 1)$	*Ranking* linear — limitado, sem singularidade de $\log$
$v_b \in [-a, a]$, $a = \operatorname{arctanh}(1 - t/T)$ — **ilimitado** em $t = 0$	$v_b = \tanh(1 - t/T)$ — limitado em $[0, \tanh 1]$	Depósitos discretos divergem sob v_b ilimitado; a saturação estabiliza
Atualização com *switching* em três casos (z / *approach* / *oscillate*)	Soma **única** $v_c\tau + v_b\Delta$ + ruído *z-branch*	Todos os ingredientes de Li em cada passo; sem sorteio de *branch* por indivíduo
Direção aleatória $X_A - X_B$	Substituída por **agregado ponderado por *ranking*** $\sum_w W_w \cdot 1[\text{agente } w \text{ usou aresta } e]$	"Diferença de dois pontos aleatórios" é indefinida em grafo

Não é uma tentativa de "melhorar" Li: são **adaptações** para preservar *feedback* ±, evolução de amplitude e *restart* em um estado discreto de feromônio.

Matriz de ablação

Duas escolhas de *design*

- **Modo de feromônio**:
  - *unified* — um τ por aresta no caminho inteiro
  - *stratified* — um τ por par (aresta, segmento entre *waypoints*)
- **Sinal**:
  - *signed* — atrai e repele (análogo completo da SMA)
  - *positive* — apenas a metade superior deposita (mais próximo de ACO)

4 condições × 4 *baselines*

	unified	*stratified*
signed	A	B
positive	C	D

Hipóteses pré-registradas

1. ***signed*** > ***positive*** em *puzzles* difíceis (*feedback* negativo quebra simetria)
2. ***stratified*** > ***unified*** quando K é grande (memória por segmento importa)
3. ZipMould > *baseline* ACO *vanilla* no *split* de teste *held-out*

Teste pareado de McNemar com correção FDR nas 4 condições × 4 baselines × seeds.

O *kernel* Numba — por que é rápido

```
@nb.njit(cache=True)
def _walker_step(walker_id, pos, visited, ..., tau, alpha, beta_log, ...):
    for d in range(4):                                  # 4 vizinhos
        nb_cell = adjacency[cur, d]
        if nb_cell < 0 or _bit_test(visited, walker_id, nb_cell): continue
        h_a = h_articulation(walker_id, nb_cell, visited, adjacency, ...)
        if h_a == NEG_INF: continue                     # desconecta subgrafo
        logits[d] = alpha * tau_val + beta_log * math.log(eta)
    # normaliza em escala log; amostra roleta; marca visita; atualiza paridade
```

- ***Bitset*** `visited` (palavras uint64) → consulta O(1), *cache-friendly*.
- `@njit` compila o *hot loop* via JIT para **velocidade de C**; Python puro é ≈100× mais lento.
- O *flood-fill* de *articulation* é o maior custo interno — `work_stack` compartilhado entre *walkers*.

Baselines + protocolo estatístico

Baseline	Feromônio	Depósito	Observações
aco-vanilla	*unsigned*, *unified*	$\propto$ *fitness*	Evaporação ρ clássica, sem ruído de *restart*
heuristic-only	*none*	—	*Greedy* apenas em η — mede a força das heurísticas
random-walk	*uniform*	*none*	Piso de exploração pura
backtracking	n/a	n/a	DFS exaustivo com poda por paridade + *articulation*

Protocolo pré-registrado

- *Splits* ***train / dev / test***, estratificados por dificuldade do *puzzle* (calculada *offline* por profundidade BFS + K/L).
- ***Held-out test set***: resultados computados *uma única vez*, depois do congelamento do *design*.
- **Teste pareado de McNemar** sobre sucesso / fracasso em cada *puzzle*.
- **Correção FDR Benjamini-Hochberg** na matriz 4×4 de condições.
- Todas as *seeds* são reprodutíveis via `derive_kernel_seed(global_seed, run_seed, puzzle_id, config_hash)`.

Visualizador — como o *trace* vira animação

```
resolvedor zipmould
    │
    ▼
registro CBOR (um estado por quadro)
    • t, v_b, v_c
    • posições dos agentes + segmentos
    • delta de tau (esparso)
    • melhor avaliação até o momento
    │
    ▼
servidor FastAPI (uv run zipmould viz serve)
    │  fluxo HTTP + cbor-x
    ▼
Vue 3 + Pinia + Tailwind 4
    • GridCanvas (SVG)
    • FitnessChart (Chart.js)
    • WalkerTable
    • controle de quadros
```

Por que CBOR

- Só τ-*deltas* **esparsos** — os *frames* continuam pequenos mesmo com 200 iterações.
- Pouco *schema*, ***streamable***, binário nativo em `cbor-x`.

Por que *replay client-side*

- O *solver* é pesado; rodamos **uma vez** para disco e depois navegamos no *browser* sem espera.
- Separa as execuções experimentais da apresentação e da inspeção.

Demo ao vivo

Indo para o visualizador...

```
# Terminal 1 — servidor HTTP do resolvedor
uv run zipmould viz serve

# Terminal 2 — servidor de desenvolvimento Vue
cd viz-web && bun run dev
```

O que vamos mostrar: carregar um *trace* · reproduzir a animação · ligar/desligar *heatmap* de τ e camadas de *walkers* · ir até uma iteração específica · comparar *signed* vs *positive* no mesmo *puzzle*.

Conclusões + trabalhos futuros

Conclusões

- A SMA de Li é um **mecanismo**, não um caminho de código: W, v_b/v_c e *z-branch* têm análogos discretos.
- *Stigmergy* (ACO) fornece o espaço de estados; SMA fornece a **dinâmica que atua sobre ele**.
- Limitar a evolução de v_b é o **maior desvio** — e foi imposto pela discretização.
- A matriz de ablação pré-registrada separa ganho de mecanismo de ganho por *tuning*.

Trabalhos futuros

- ***Warm-start*** com a *baseline heuristic-only*.
- ***Learned heuristic*** η: MLP local junto a Manhattan/Warnsdorff/etc.
- ***Multi-objective***: caminho curto sob orçamento de paredes.
- **Além do Zip**: *knight's tours* e *Hamiltonian-path-with-pinned-vertices*.

Obrigado

Perguntas são bem-vindas — especialmente as céticas.

Referências

- **Li, S., Chen, H., Wang, M., Heidari, A. A. & Mirjalili, S.** (2020). *Slime mould algorithm: A new method for stochastic optimization*. *Future Generation Computer Systems* 111, 300–323.
- **Dorigo, M. & Stützle, T.** (2004). *Ant Colony Optimization*. MIT Press.
- **Mirjalili, S. & Lewis, A.** (2016). *The Whale Optimization Algorithm*. *Advances in Engineering Software* 95, 51–67.
- **Tero, A. et al.** (2010). *Rules for Biologically Inspired Adaptive Network Design*. *Science* 327, 439–442.
- **Warnsdorff, H. C.** (1823). *Des Rösselsprungs einfachste und allgemeinste Lösung*.
- **McNemar, Q.** (1947). *Note on the sampling error of the difference between correlated proportions or percentages*. *Psychometrika* 12, 153–157.